

Analysis and Design of Algorithm

Project II

Bonus Documentation

Ahmed Mohammed

May 2026

1 Optimization for Mid-Week Events

1.1 Original Approach and the Computational Bottleneck

In our initial implementation, the system processed dynamic mid-week events (cancellations and reschedules) by updating the system state and subsequently rerunning the matching algorithm. Because our matching utilizes the Hungarian Algorithm to satisfy the lexicographic objective function, it operates with a cubic time complexity of $O(\max(R, S)^3)$, where R is the number of pending requests and S is the number of available slots.

Running an $O(N^3)$ algorithm for every minor dynamic event introduced a severe computational bottleneck. If there are T dynamic events throughout the week, the original time complexity scaled to $O(W \cdot T \cdot \max(R, S)^3)$.

1.2 Theoretical Insight: The Locked State

To optimize the system, I analyzed the bipartite graph during the mid-week simulation. A crucial realization is that **past assignments are locked**.

At any point mid-week, let P be the set of currently Pending requests and A be the set of currently Available slots. Because the Hungarian algorithm was already executed at the start of the week, we are mathematically guaranteed that **no compatible pairs exist between P and A** . If a compatible pair existed, the algorithm would have matched them during the previous run.

Therefore, when a dynamic event introduces newly freed resources, the resulting bipartite graph is extremely sparse, allowing us to bypass the $O(N^3)$ and replace it with mathematically equivalent $O(N)$ linear scans.

1.3 Optimized $O(N)$ Event Handlers

Relying on the disjoint nature of P and A , I redesigned the event handlers to perform greedy matching:

1.3.1 1. Request and Slot Cancellations (Star Graph Optimization)

When a matched request is canceled, its corresponding mentor slot is returned to the A pool. Because the rest of P and A share no valid edges, the only valid edges in the entire system must connect to this *single newly freed slot*.

In graph theory, this topology is known as a **Star Graph**. The maximum possible cardinality for a bipartite matching on a Star Graph is exactly 1. Therefore, running the Hungarian algorithm is unnecessary. We achieve the exact same optimal result by iterating through P in $O(N)$ time and assigning the single request that yields the highest lexicographic benefit score for that specific slot. The inverse logic applies identically in $O(N)$ time when an assigned slot is canceled, freeing a single request.

1.3.2 2. Slot Rescheduling

Rescheduling an assigned slot breaks an existing match, simultaneously freeing one request (r_{orphan}) into P and one slot (s_{new}) into A . The maximum possible new assignments we can extract from this state is exactly 2. We resolve this optimally in linear time via the following logic:

1. **Independent Scans:** In $O(N)$ time, scan A to find the best compatible slot for r_{orphan} (yields s_{best}). Separately, scan P to find the best compatible request for s_{new} (yields r_{best}).
2. **Option A (Two Assignments):** If both s_{best} and r_{best} exist, we assign $(r_{\text{orphan}} \rightarrow s_{\text{best}})$ and $(r_{\text{best}} \rightarrow s_{\text{new}})$. This safely achieves the maximum cardinality of 2.
3. **Option B (One Assignment):** If Option A is impossible, the maximum cardinality is 1. We simply evaluate the valid permutations: $(r_{\text{orphan}} \rightarrow s_{\text{new}})$ (a self-match at the new time), $(r_{\text{orphan}} \rightarrow s_{\text{best}})$, or $(r_{\text{best}} \rightarrow s_{\text{new}})$, and greedily select the single assignment that maximizes the benefit score.

1.4 Time Complexity Improvement

We reduced the number of times we rerun the Hungarian algorithm.

- **Old Complexity:** $O(W \cdot T \cdot \max(R, S)^3)$
- **New Complexity:** $O(W \cdot \max(R, S)^3 + W \cdot T \cdot \max(R, S))$

The $O(N^3)$ Hungarian matching is now strictly executed **only once per week** (at the start of the week). The T dynamic events are handled entirely via linear scans ($O(R + S)$).

2 Contribution and Source Code

This part was done completely and individually by Ahmed Mohammed. You can find the source code in the same repo at `bonus/ahmed`.

Note: The optimization was only tested on the same tests generated for the original solution.